

Úvod do databázových systémů

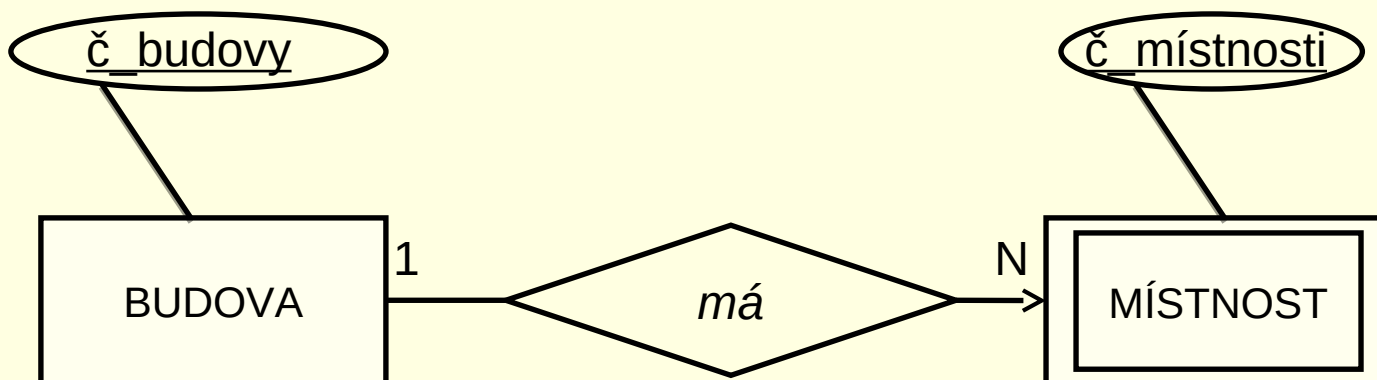
Kapitola 4
Konceptuální modelování

Slabé entitní typy

- součástí klíče některých entit nemusí být pouze jejich vlastní atributy
 - entita není jednoznačně identifikovatelná pouze pomocí svých atributů, mohou existovat instance, které mají stejné hodnoty atributů
 - např. identifikace místnosti je možná pouze ve spojení s identifikací budovy, v němž se nalézá
 - budova – identifikační vlastník
 - místnost – slabý entitní typ
 - vztah budova-místnost – identifikační vztah
- slabý entitní typ má **vždy** povinné členství v identifikačním vztahu

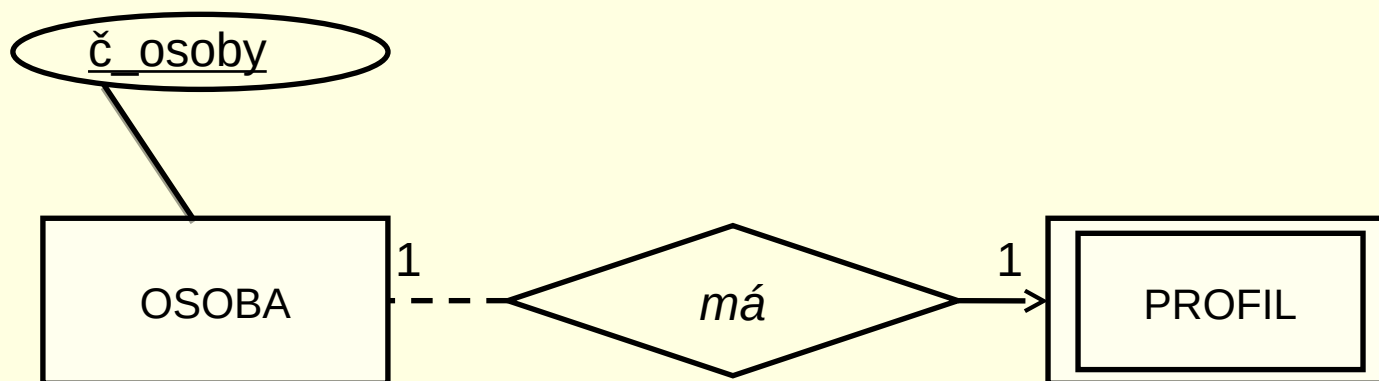
Slabé entitní typy

- příklad: v databázi může existovat více místností s týmž číslem, které se však nalézají v různých budovách
 - identifikační klíč je dvojice (č_budovy, č_místnosti)
 - vlastní atribut „č_místnosti“ je jen **částečný klíč**



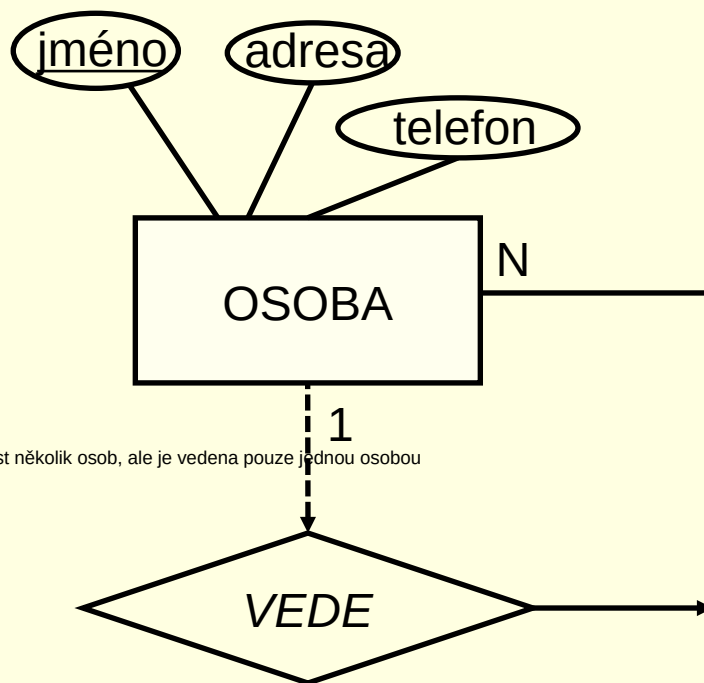
Slabé entitní typy

- někdy nemusí mít slabá entita žádný částečný klíč
 - pak jde v podstatě o vztah 1:1
 - každý identifikační vlastník pak může mít jen jednu instanci identifikačně závislé entity
 - každá osoba může mít nejvýše jeden profil



Rekurzivní typ vztahu

typ entity vstupuje do vztahu sám se sebou



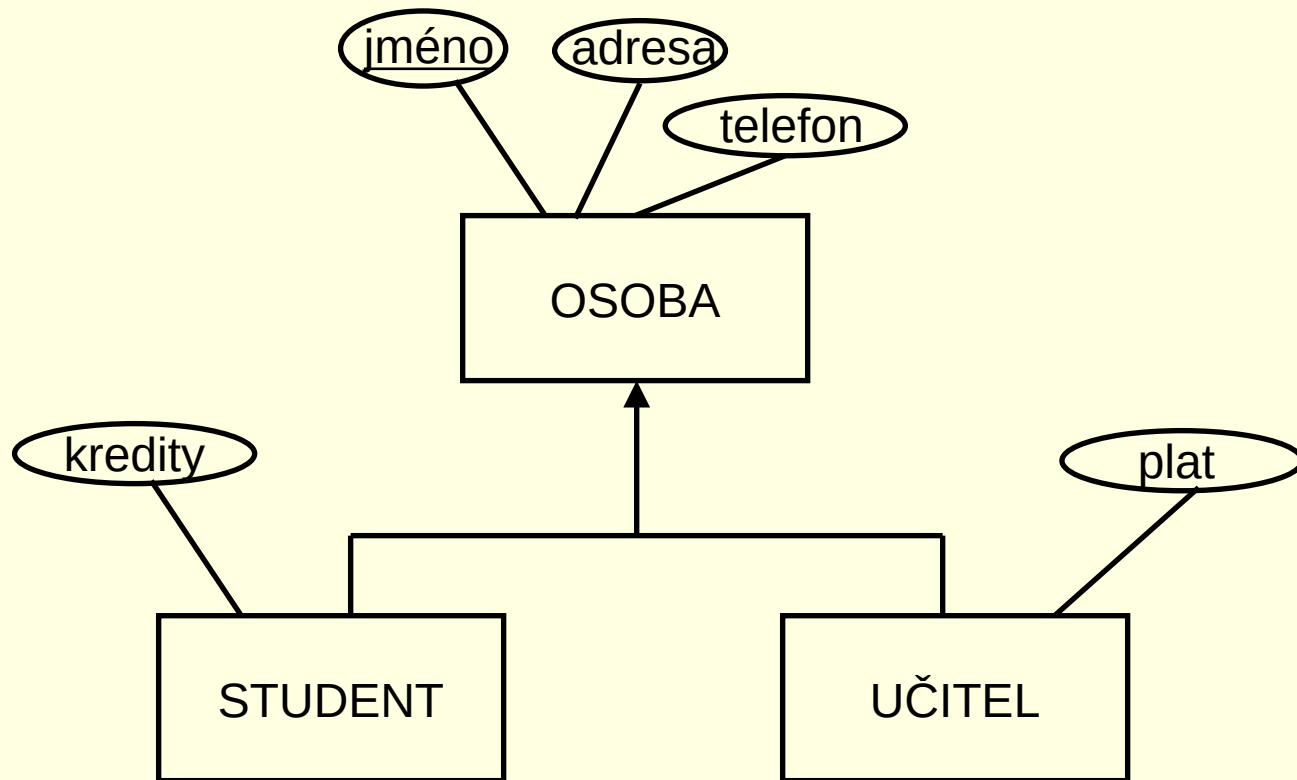
vzniká stromová hierarchie – osoba může vést několik osob, ale je vedena pouze jednou osobou

je vhodné zavést tzv. **role** (vede, je_veden)

ISA hierarchie

- analogie dědičnosti v OOP
 - např. OSOBA může být buď STUDENT nebo UČITEL – entitní podtyp
 - **STUDENT IS A OSOBA, UČITEL IS A OSOBA**
 - společné atributy: jméno, RČ, adresa
 - atributy vázané pouze na podtyp
 - STUDENT: počet kreditů, ...
 - UČITEL: platová třída, ...
 - ISA vztah je reflexivní a tranzitivní, zavádí do schematu obvykle stromové struktury

ISA hierarchie



ISA hierarchie

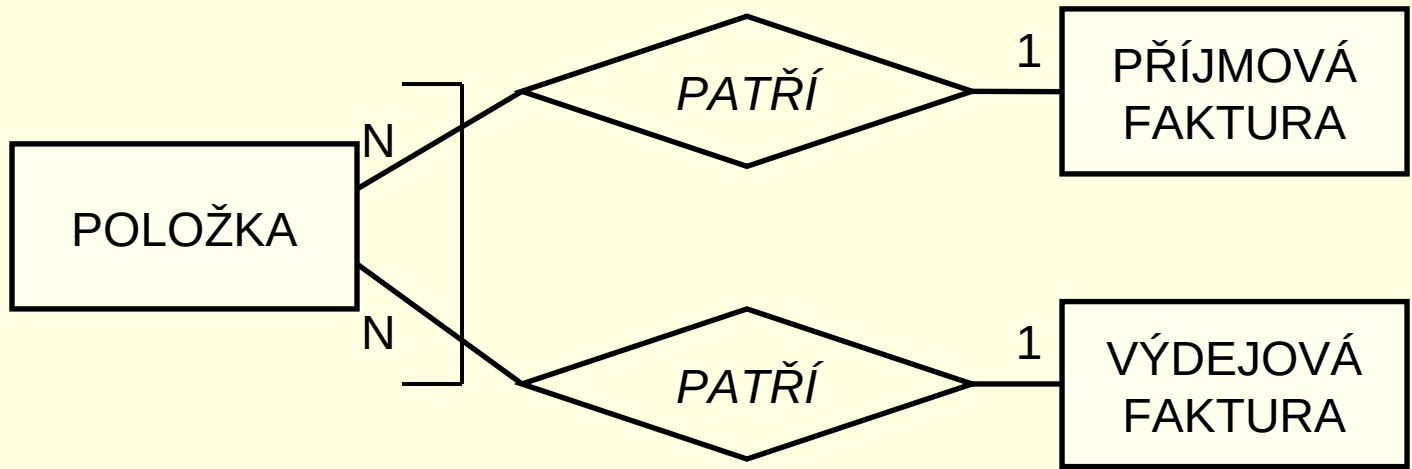
- důležité je si uvědomit, že entitní podtypy musí tvořit úplné pokrytí entitního nadtypu,
 - v našem případě každá osoba je buď student nebo učitel, ale nemůže být nic jiného
- naopak platí, že prvek entitního nadtypu může patřit pouze do jednoho ent. podtypu, tj. entitní podtypy jsou vzájemně disjunktní
 - příslušnost k entitním podtypům je výlučná
 - tedy když je nějaká osoba studentem, nemůže být zároveň učitelem a naopak
 - to je někdy omezující a v rozporu s realitou, proto je např. u učitele a studenta lepší zavést místo ISA hierarchie identifikační závislost na entitě osoba

Speciální typy atributů

- konceptuální model se nemusí omezovat pouze na atomické atributy (logický model se na ně obvykle již omezuje)
- skupinový atribut
 - např. adresa (ulice, číslo, město, PSČ)
 - heterogenní strukturovaný datový typ
 - může tvořit hierarchie
 - užitečný, přistupujeme-li někdy ke složkám a někdy k celku
- vícehodnotový atribut
 - např. atribut telefon ent. typu ZAMĚSTNANEC
 - když předem nevíme, kolik prvků bude mít

Výlučný typ vztahu

- entitní typ vstupuje právě do jednoho z N vztahů

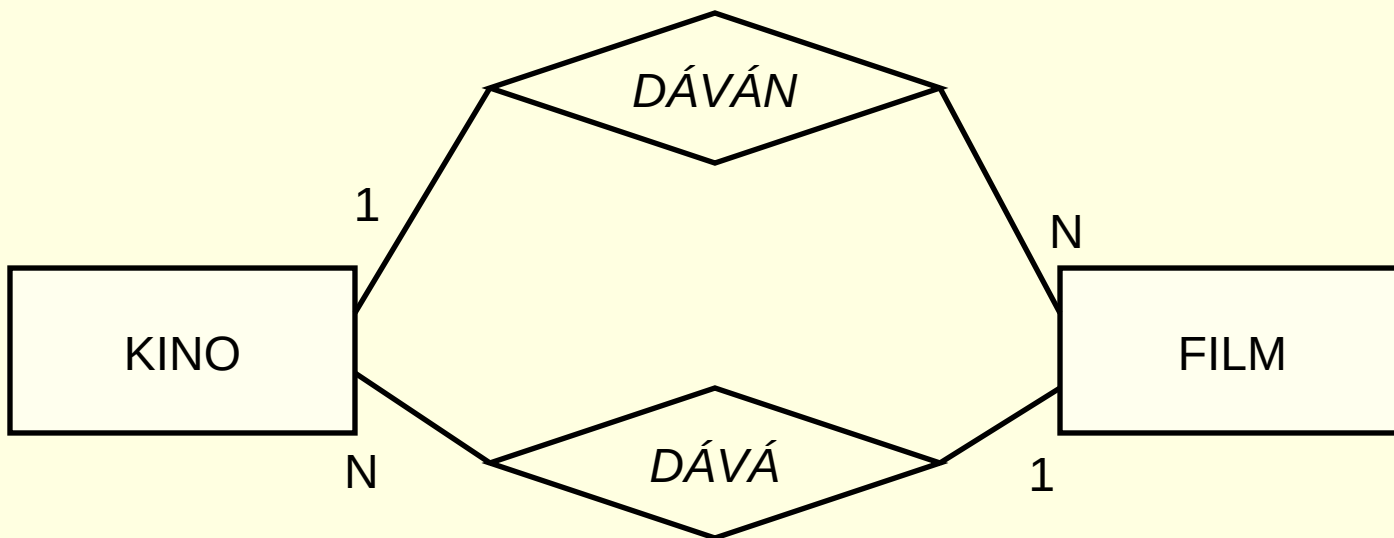
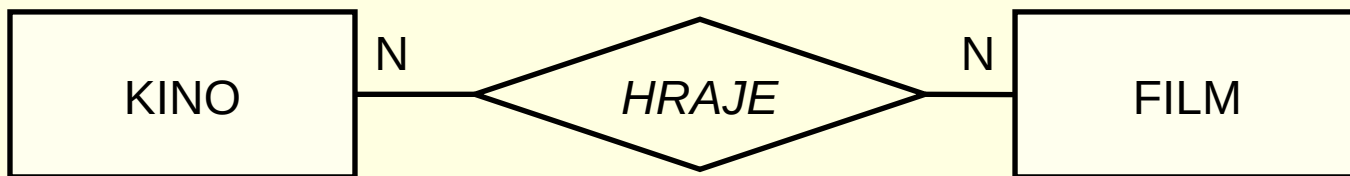


Dekompozice M:N vztahu

- návrh konceptuálního schematu je nezávislý na logickém modelu
 - to může mimo jiné znamenat, že ne všechny konstrukty konceptuálního modelu jdou přímo převést do modelu logického
 - přesto není třeba se těmito konstruktům vyhýbat
- například relační SŘBD neumějí přímo reprezentovat vztahy M:N
 - vztahy M:N je určitě vhodné používat při konceptuálním modelování, avšak musíme být schopni je následně rozdělit do dvou vztahů typu 1:N – dekompozice

Dekompozice M:N vztahu

- chybná dekompozice

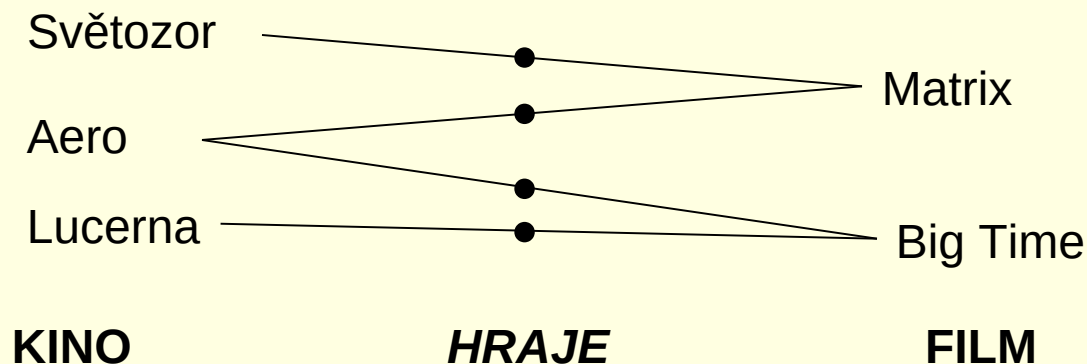


Dekompozice M:N vztahu

- chybná dekompozice
 - původní schema říká, že ani v jednom směru neexistuje funkční závislost mezi účastníky vztahu, ani jedna entita není determinantem vztahu
 - nové schema
 - ve vztahu *DÁVÁ* je entita KINO determinantem vztahu, určuje tedy jednoznačně promítaný film
 - ve vztahu *DÁVÁN* je entita FILM determinantem vztahu, určuje tedy jednoznačně kino, kde se hraje
 - výše uvedené podmínky jsou současně splněny pouze pro vztah 1:1, což je jen podmnožina vztahu M:N

Dekompozice M:N vztahu

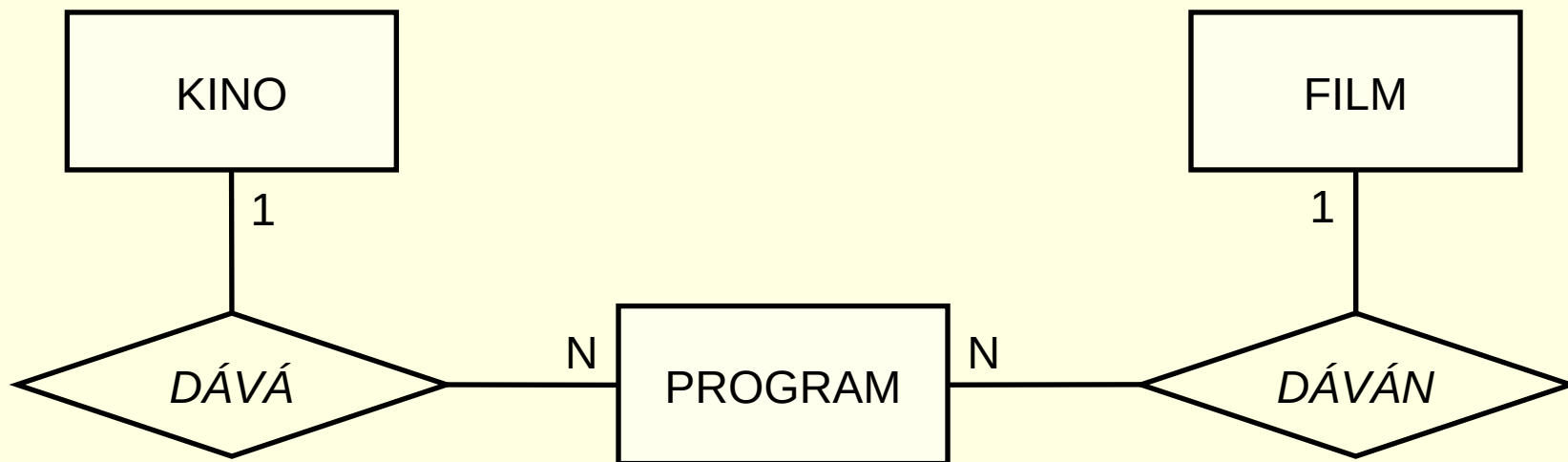
- podívejme se na diagram výskytů



- každý výskyt vztahu odpovídá promítání konkrétního filmu v konkrétním kině
 - budeme-li považovat **výskyt vztahu** za **výskyt entitního typu**, máme vyhráno
 - vztah lze v podstatě vždy považovat za entitu
 - průnikový entitní typ**

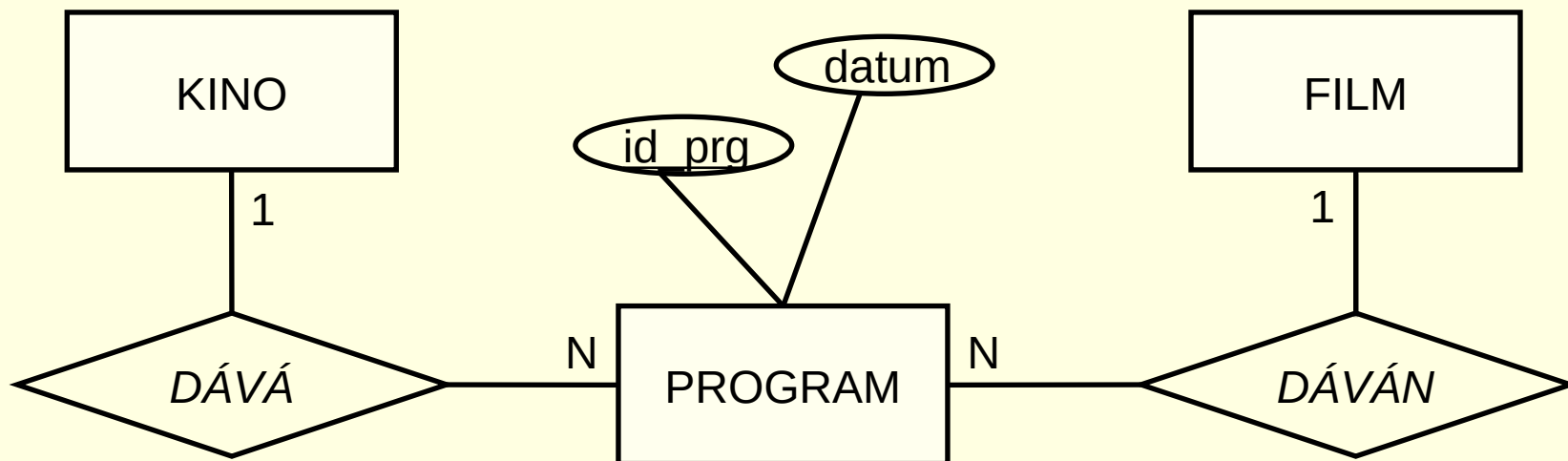
Dekompozice M:N vztahu

- správný princip dekompozice



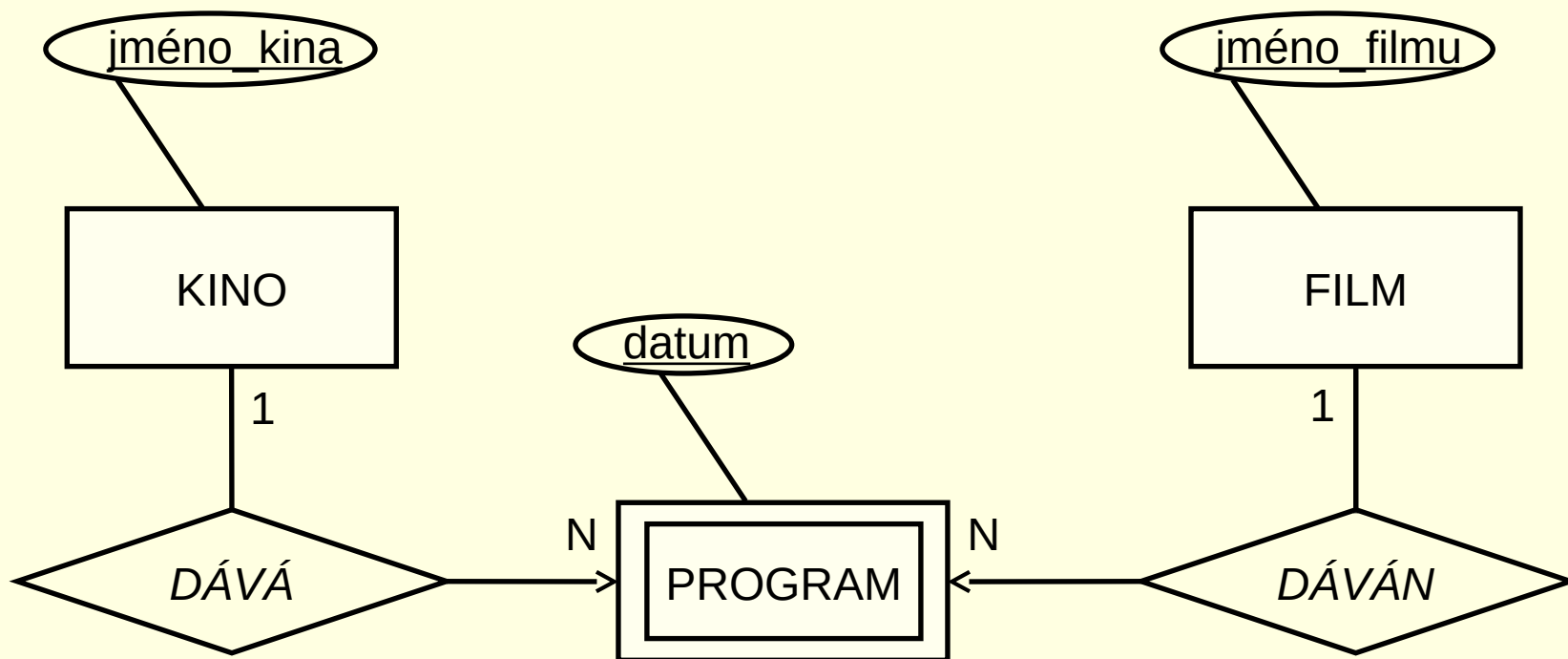
Dekompozice M:N vztahu

- jak identifikovat novou entitu?
 - buď to bude regulérní silná entita s vlastním identifikátorem (umělým...)



Dekompozice M:N vztahu

- jak identifikovat novou entitu?
 - nebo lze využít identifikační závislosti
 - PROGRAM má identifikátor (jméno_kina, jméno_filmu)
 - pokud není atribut „datum“ klíčový, kino nemůže promítat jeden film v různých dnech



Korektní konceptuální schema

- má smysluplnou a jednoznačnou sémantiku
 - ISA hierarchie a identifikační vztahy přinášejí potíže
 - např. pokud má entitní podtyp dva předchůdce (vícenásobná dědičnost), nevznikne strom, ale složitější graf, který je obtížné reprezentovat
 - jak identifikovat podtyp, který má dva rodiče?
 - zavádí se pravidlo, že v ISA hierarchii musí být na vrcholové úrovni pouze jeden uzel (entita) – **zdroj**
 - pak lze snadno dědit identifikační klíč zdroje

Korektní konceptuální schema

1. každý entitní typ má max. jeden zdroj ISA hierarchie
2. ISA vztahy netvoří orientovaný cyklus
 - cyklus by popisoval tytéž množiny => redundance
3. identifikační vztahy netvoří orientovaný cyklus
 - entita by identifikovala sebe sama
4. typ entity v ISA hierarchii, který není zdrojem, není identifikačně závislý na jiném ent. typu
 - je již identifikován zdrojem své ISA hierarchie
5. jména typů entit a vztahů jsou v rámci schematu jednoznačná, jména atributů jednoznačná v rámci objektu
6. vystupuje-li entita ve vztahu víckrát, každé členství je popsáno dalším identifikátorem – rolí

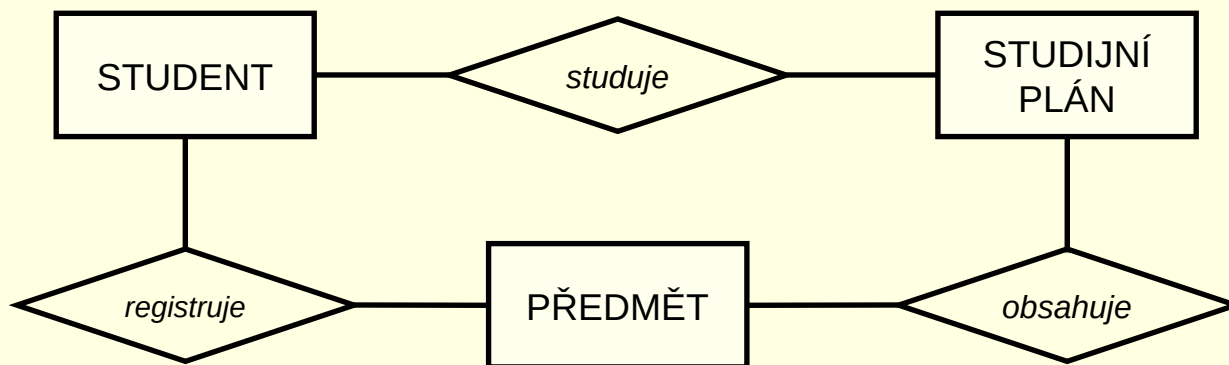
Korektní konceptuální schema

analogií zdroje ISA hierarchie je identifikační zdroj

8. vstupuje-li slabá entita do dvou identifikačních vztahů, pak
 - buď pro ni existují dva identifikační zdroje
 - nebo týž id. zdroj vystupuje ve dvou rolích, jež se musí odlišit označením
 - např. máme entitu ŽÁDOST_O_MANŽELSKOU_PŮJČKU a OSOBA, entita OSOBA vystupuje ve dvou rolích
- splňuje-li konceptuální schema body 1-8, jedná se o **dobře definované** schema

Korektní konceptuální schema

- pozor též na cykly v konceptuálním modelu
 - konceptuální schema jako takové může být korektní, ale bez dodefinování patřičných integritních omezení bude umožňovat ukládat do databáze rozporuplná data
 - mějme příkladně část schematu typu



- podle předpisu student studuje určitý studijní plán a smí si registrovat pouze předměty tohoto studijního plánu
- tento předpis však konceptuální schema NIJAK NEVYJADŘUJE
 - obě větve (spodní a horní) jsou na sobě nezávislé: student může být ve vztahu s předmětem, který není obsažen ve studijním plánu, jež student studuje
 - je třeba dodefinovat předpis ve formě samostatného IO

Konceptuální analýza

- z reality je třeba do modelu zobrazit podstatné jevy a to pokud možno optimálním způsobem vzhledem k plánovanému použití systému
 - zvolená reprezentace reality v modelu závisí na schopnostech a vidění analytika
 - stejné jevy lze modelovat různým způsobem
 - viděli jsme, že rozlišení entita – vztah není vždy jasné
 - také rozlišení atribut - vztah je často nejasné, v obou případech jde o funkci

Konceptuální analýza

- př. evidujeme osoby, které sedí v místnostech
 - OSOBA má řadu atributů, jde určitě o samostatný entitní typ
 - souvislost osoby s místností může být vyjádřena
 1. číslo místnosti přidáme jako další atribut OSOBY
 2. zavedeme entitní typ MÍSTNOST a typ vztahu *SEDÍ*
 - první způsob lze akceptovat, pokud nás nezajímají další atributy místnosti
 - jakmile se u místnosti evidují další vlastnosti, jako třeba telefonní linka, kapacita a možnost vstupu na čip, určitě použijeme druhý způsob, protože „kapacita“ jistě není vlastností OSOBY

Konceptuální analýza

- př. evidujeme osoby, které sedí v místnostech
 - vhodná modelová reprezentace není samoučelná, špatný návrh nám totiž může znemožnit určité typy dotazů
 - např. zvolíme-li v tomto příkladu způsob 1, nejsme schopni evidovat místnosti, ve kterých zrovna nikdo nesedí
 - na úrovni logického modelu (relačního) se tyto anomálie řeší tzv. **normalizací** (viz další kapitoly)

Metodika návrhu databáze

- první krok
 - shromažďování a formulace uživatelských požadavků (diskuse se zaměstnanci)
 - identifikace a dokumentace dat, která je třeba reprezentovat
 - zde se vynořují entity a vztahy, hrubá datová a funkční analýza
- přestože se zde funkční analýzou nezabýváme, je nutné se o ní zmínit, neboť povědomí o procesech, které budou databázi využívat, je klíčové pro optimální návrh schématu

Metodika návrhu databáze

- rozpracujeme **pracovní postupy**, které provádějí **typičtí pracovníci**, rozhodnout, zda dostanou počítačovou podporu
- z typických pracovníků se později stanou uživatelské **role** (třídy uživatelů)
- identifikace **událostí**, které spouštějí pracovní postupy
- vlastní E-R modelování

Metodika návrhu databáze

- vlastní E-R modelování
 - definice typů nezávislých entit, pro začátek stačí navrhnout identifikátory, další atributy počkají
 - definice typů vztahů
 - definice typů závislých entit
 - přiřazení atributů entitním a vztahovým typům
- proces by měl probíhat tak dlouho, dokud nedojde k souhlasu modelu s představou zadavatele

Příklad konceptuálního modelu

■ zadání – databáze psího útulku

- Chceme evidovat psy, kteří jsou chováni v kotcích v útulku. Útulek je dělen na pavilony, v každém pavilonu se může nalézat více kotců. V jednom kotci může být víc psů. Každý evidovaný pes musí mít přiřazen kotec, určitý kotec ale může být prázdný. Dále evidujeme ošetřovatele, každý ošetřovatel má na starost několik kotců, každý kotec má jednoho ošetřovatele. U psa chceme evidovat: číslo známky, přibližnou rasu, přibližný věk, stručně popis. U kotce chceme evidovat: číslo (unikátní v rámci pavilonu), kapacitu, typ. U pavilonu chceme evidovat: id. kód, umístění, popis. U ošetřovatele evidujeme: jméno, rodné číslo, pohlaví. Zajímá nás i historie, kde kdy byl který pes.

Příklad konceptuálního modelu

- první krok – identifikace entitních typů a klíčů
 - **PES** – ***evidenční číslo*** (číslo známky by mohlo být kandidátem na klíč, ale co např. se psy, kteří nemají známku? známka také nemusí být zcela unikátní pro všechny existující psy)
 - **KOTEC** – ***číslo*** (můžeme si dovolit, neočekáváme přečíslovávání kotců)
 - **PAVILON** – ***id. kód*** (můžeme, neočekáváme změny kódů pavilonů)
 - **OŠETŘOVATEL** – ***osobní číslo*** (rodné číslo by mohlo být kandidátem na klíč, má ale nevhodný datový typ – řetězec – a navíc u cizinců se každý půlrok mění a nemusí být zcela unikátní)

Příklad konceptuálního modelu

- druhý krok – identifikace vztahů
 - „bydlí“ mezi entitami PES a KOTEC
 - M:N
 - „umístěn“ mezi entitami PAVILON a KOTEC
 - 1:N
 - „stará se“ mezi ent. OŠETŘOVATEL a KOTEC
 - 1:N
- třetí krok – identifikace slabých entitních typů
 - KOTEC – závisí na entitě PAVILON

Příklad konceptuálního modelu

■ čtvrtý krok – přiřazení atributů

■ PES

- ev. číslo (celé číslo, nejlépe automatické)
- číslo známky (celé číslo větší než 0)
- rasa (text max 255 znaků),
- rok narození (celé číslo větší než 1900)
- popis (text neomezený)

■ KOTEC

- číslo (celé číslo > 0)
- kapacita (celé číslo > 0)
- typ (text neomezený)

Příklad konceptuálního modelu

- čtvrtý krok – přiřazení atributů

- PAVILON

- id. kód (řetězec max. 3 znaky)
 - umístění (text max 255 znaků)
 - popis (text neomezený)

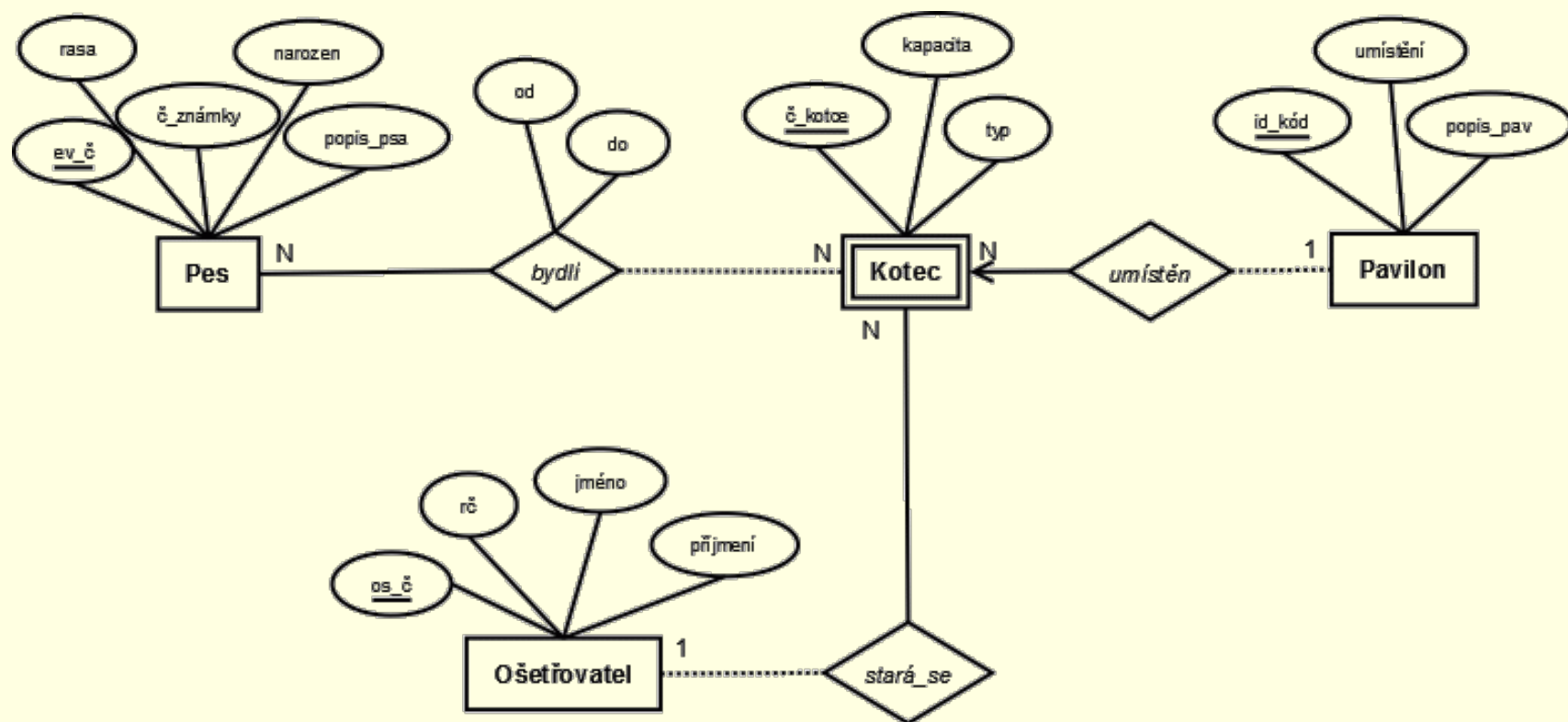
- OŠETŘOVATEL

- os. číslo (celé číslo, nejlépe automatické)
 - rodné číslo (řetězec 9 nebo 10 znaků, dělitelný 11)
 - jméno (text max 30 znaků)
 - příjmení (text max 50 znaků)

- vztah *bydlí*

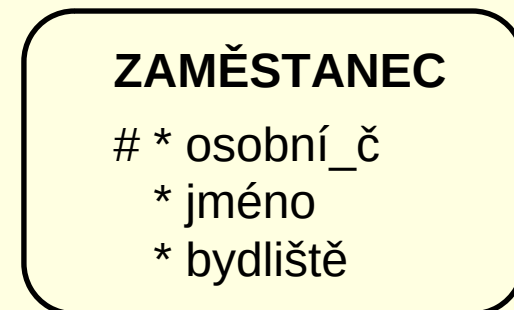
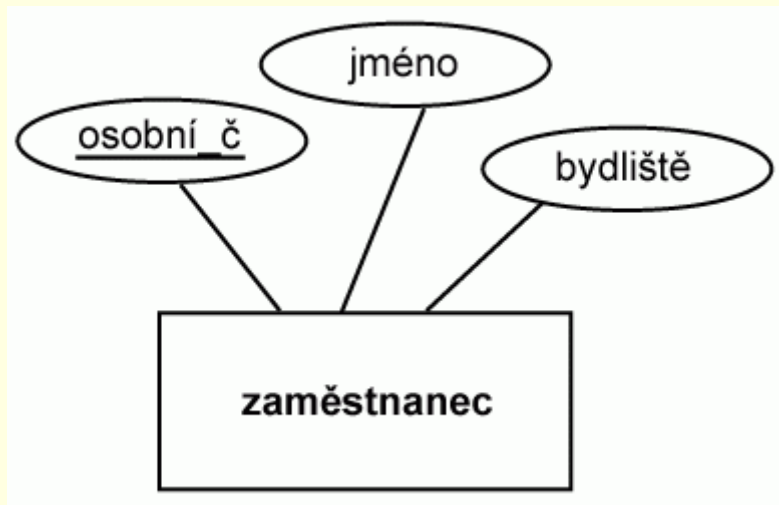
- od (datum)
 - do (datum)

Příklad konceptuálního modelu



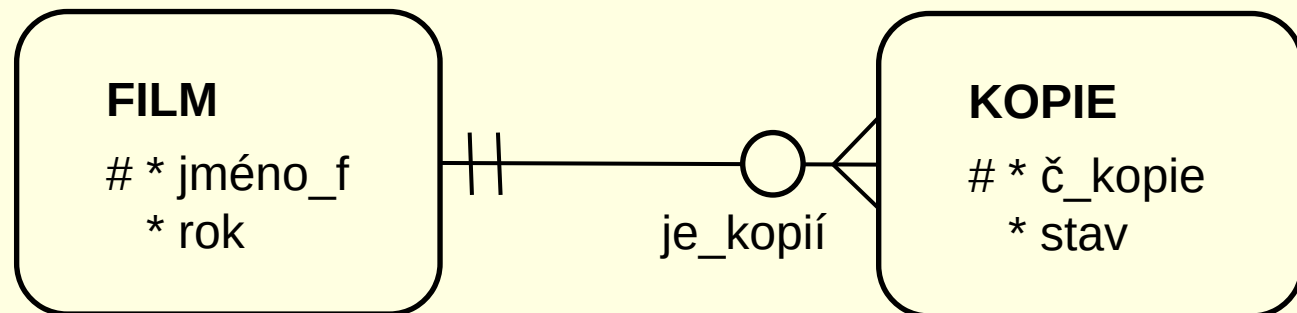
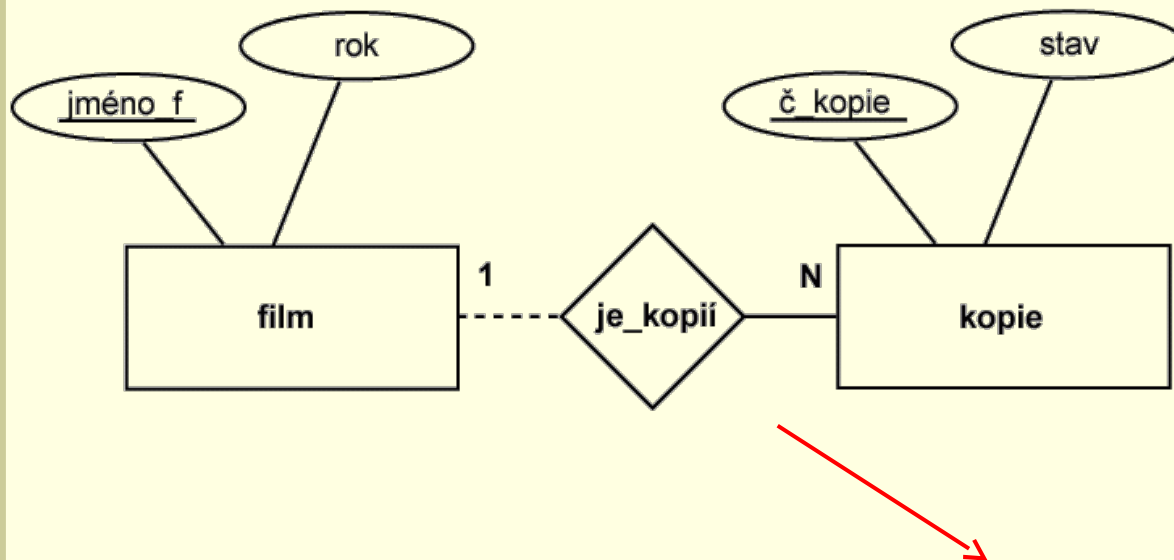
Martinova notace E-R diagramu

- doposud používaná notace: Chenova
- často je používána Martinova crow's foot notace
 - entita s atributy a klíčem



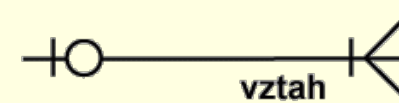
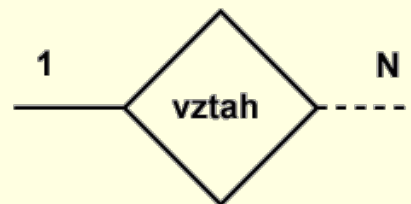
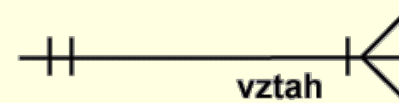
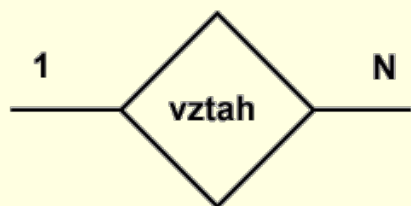
Martinova notace E-R diagramu

- dva entitní typy a vztah



Martinova notace E-R diagramu

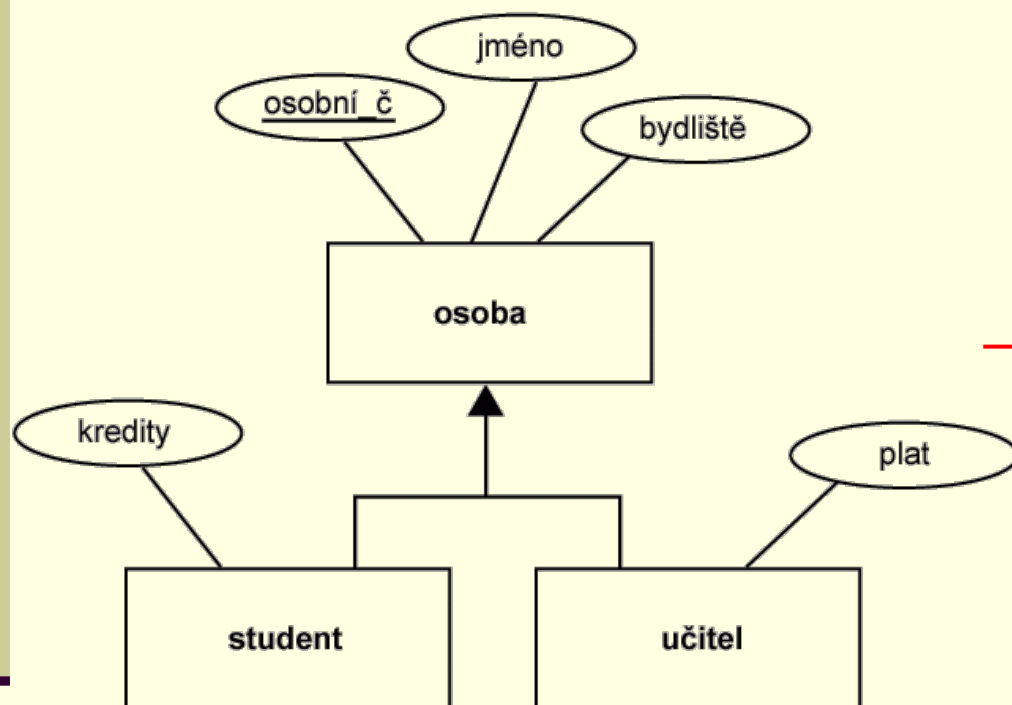
- vztahy s různými kardinalitami a parcialitami



- pozor, Martinova notace (ani UML Class Diagram) neumožňuje definovat atributy vztahu

Martinova notace E-R diagramu

■ ISA hierarchie



OSOBA

* osobní_č
* jméno
* bydliště

STUDENT

* kredity

UČITEL

* plat

Martinova notace E-R diagramu

- Martinova notace umožňuje specifikovat i nepovinné atributy

ENTITA

- # * identifikátor
- * povinný_atribut
- ° nepovinný_atribut